

< 200 CONCRETE OPEN PROBLEMS IN MECHANISTIC INTERPRETABILITY >

200 COP in MI: Interpreting Reinforcement Learning

by **Neel Nanda** 10th Jan 2023

This is the ninth post in a sequence called 200 Concrete Open Problems in Mechanistic Interpretability. [Start here](#), then read in any order. If you want to learn the basics before you think about open problems, check out [my post on getting started](#). Look up jargon in [my Mechanistic Interpretability Explainer](#)

Motivating papers: [Acquisition of Chess Knowledge in AlphaZero](#), [Understanding RL Vision](#)

Disclaimer: *My area of expertise is language model interpretability, not reinforcement learning. These are fairly uninformed takes and I'm sure I'm missing a lot of context and relevant prior work, but this is hopefully more useful than nothing!*

Motivation

Reinforcement learning is the study of how to create agents - models that can act in an environment and form strategies to get high reward. I think there are a lot of deep confusions we have about how RL systems work and how they learn, and that trying to reverse engineer these systems could teach us a lot! Some high-level questions about RL that I'd love to get clarity on:

- How much do RL agents learn to **actually model their environment** vs just being a big bundle of learned heuristics?

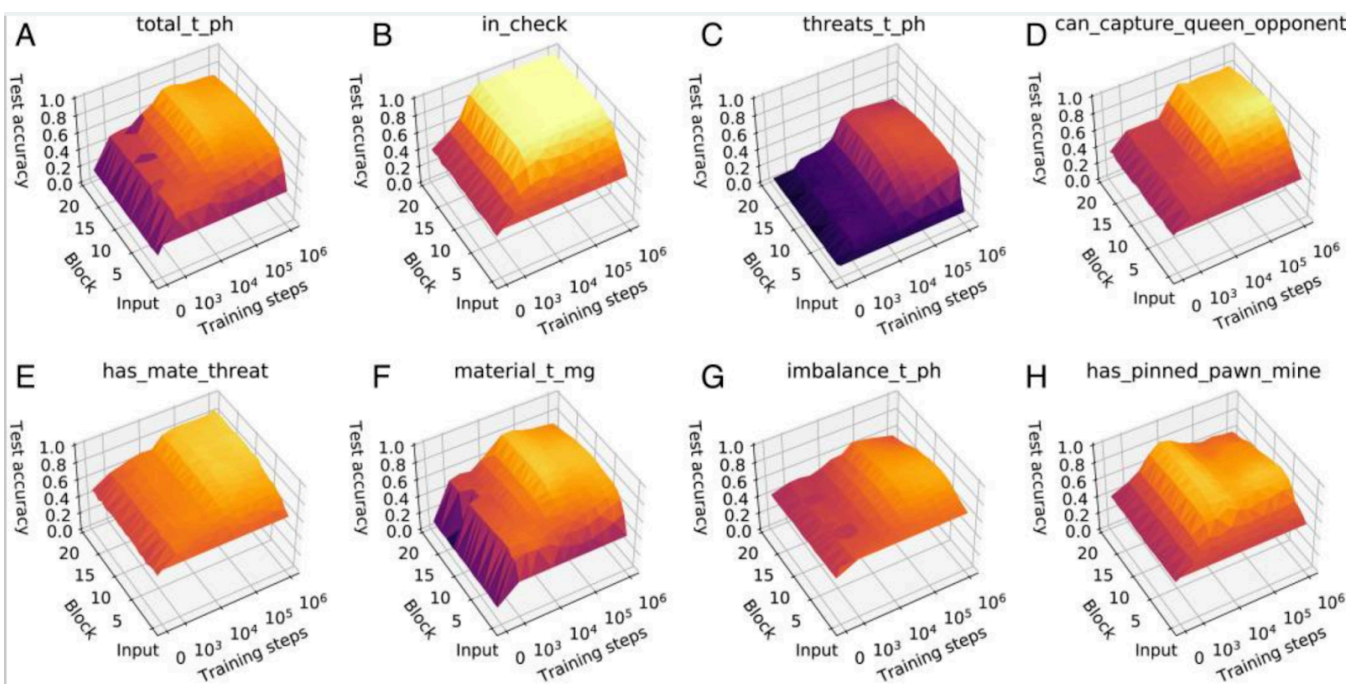
- In particular, how strong a difference is there between model-free and model-based systems here?
- Do models explicitly **reason about their future actions**? What would a circuit for this even look like?
- What are the **training dynamics** of an RL system like? What does its path to an eventual solution look like? A core hard part about training RL systems is that the best strategies tend to require multi-step plans, but taking the first step only makes sense if you know that you'll do the next few steps correctly.
 - What does it look like on a circuit level when a model successfully explores and finds a new and better solution?
- **How much do the lessons from mechanistic interpretability transfer**? Can we even reverse engineer RL systems at all?
 - How well do our conceptual frameworks for thinking about networks transfer to RL? Are there any things that clearly break, or new confusions that need to be grappled with?
 - I expect training dynamics to be a particularly thorny issue - rather than learning a fixed data distribution, whether an action is a good idea depends on which actions the *model* will take in future.

Further, I think that it's fairly likely that, whatever AGI looks like, it involves a significant component of RL. And that training human-level and beyond systems to find creative solutions to optimise some reward is particularly likely to result in dangerous behaviour. Eg, where models learn the instrumental goals to seek power or to deceive us, or just more generally learn to represent and pursue sophisticated long term goals and to successfully plan towards these.

But as is, much alignment work here is fairly speculative and high level, or focused on analysing model behaviour and forming inferences. I would *love* to see this grounded in a richer understanding of what is *actually* going on inside the system. Some more specific questions around alignment that I'm keen to get clarity on:

- Do models actually learn **internal representations of goals**? Can we identify these?
 - I'm particularly interested in settings with **robust capabilities but misgeneralised goals** - what's going on here? Can we find any evidence for or against **inner optimisation concerns**?
- How much are models **actually forming plans internally**? Can we identify this?
- Do models ever **simulate other agents**? (Especially in a multi-agent setting)

A work I find particularly inspiring here is Tom McGrath et al's work, [Acquisition of Chess Knowledge in AlphaZero](#). They used linear probes to look for which chess concepts were represented within the network (and had the fun addition of having a former world chess champion as co-author, who gave commentary on the model's opening play!). And they found that, despite AlphaZero being trained with no knowledge of human chess playing, it had re-derived many human chess concepts. Further, by applying their probes during training, they found that many of these concepts arose in a sudden phase transition, and were able to contrast the order of concepts learned to the history of *human* chess playing. I think this is extremely cool work, and I'd love to see extensions that try to really push on the angle of reverse-engineering and learning from a superhuman system, or on trying to reverse-engineer *how* some of these concepts are computed.



Overall, I think it's pretty obvious that making progress reverse-engineering RL would be important! But is it tractable? I find it hard to say - I just haven't seen that much work on it! I personally expect us to be able to make *some* progress in reverse-engineering RL, but for it to be even harder than reverse-engineering normal networks. Deep RL systems are, fundamentally, neural networks, and the same kind of techniques should work on them. But there are also a bunch of weirder things about RL, and I don't know what roadblocks will appear.

In my opinion, the lowest hanging fruit here is to just really grapple with reverse engineering even a simple system trained with RL, and to see how deep an understanding we can get.

Tips

- RL is complicated! To start out, I'd aim to cut down the complexity as much as possible
 - **Aim for simpler problems first:**
 - Try to have discrete action and state spaces. This is simpler, easier to reason about, easier to construct counter-factual inputs for, etc.
 - Caveat - this *may* not work, and some simple problems (eg Cartpole) may be too toy to be interesting
 - Problems where you only need to know the current state rather than the full history (Markov Decision Processes aka MDPs)
 - Try to think through what kind of algorithms the final model might learn, and how interesting/interpretable this might be. If it's just going to eg memorise a fixed series of steps, then it's probably too simple to be interesting
 - Try to train and interpret **the smallest model that works** (this may be significantly smaller than the paper's!)
 - Speculatively, this may be easiest for a small transformer - I have a fuzzy intuition that the sequence based structure of transformers tend to more naturally represent discrete logical structure than conv nets, and attention makes it easier to observe the flow of information through the network. I further speculate that discrete input spaces makes things cleaner and easier to interpret.
 - This is a weakly held take, it's certainly possible that smaller models are harder because they eg have more superposition and more confusing abstractions! Generally, small language models were easier and small image models were harder (than larger models).
- A good starting point to interpret a model is **finding some counterfactual** - two inputs that are mostly similar, but where the model takes importantly different actions. This can let you control for lots of irrelevant model behaviour, and isolate out the circuit that matters, using techniques like [activation patching](#)
- I would mentally divide a network into 3 parts - **sensory neurons -> analysis -> motor neurons**
 - The “sensory neurons” take in the raw data and process it (eg mapping the pixels of a Pong game to features like the position and velocity of the ball)
 - Logic to actually analyse these features, potentially including models of itself or the world, to figure out what to do
 - “Motor neuron” to then convert these concepts to actual actions

- This is a useful distinction to have in mind, because conceptually these are different circuits of the network
 - Ideally, they would also be different layers of the network, but things are often messy - eg *some* of the sensory stuff is in layer 1, the rest is in layer 2, but *some* of layer 2 is doing conceptual stuff with the sensory stuff that's already there
- Note: This framing applies to models in general, not just RL!
- **Training dynamics based projects**
 - If training your own system, make sure to have code that can be re-run deterministically (ie, explicitly setting a random seed), and to take many checkpoints
 - Focus on understanding the final, fully trained model, before studying it during training. You don't need to *fully* reverse-engineer it, but it's very useful to be confident you understand the parts of the behaviour you care about, and ideally to have distilled these to key patterns or metrics, before you try to extrapolate this to the system during training
 - Further, I'm much more comfortable using somewhat janky/unreliable but automated methods if they agree with the results of more rigorous methods on the final model.
 - See all the tips in [the training dynamics post](#)!
- **Expect interpreting RL to be hard!** In particular, I expect there to be a lot of rabbit holes and weird anomalies. If you want to make research progress, it's important to prioritise, and to keep a clear picture in mind re what is actually *interesting* about your exploration.
 - Though random exploration can also be useful, especially when starting out or just messing around! It's important to balance exploring and exploiting. I try to give myself a bounded period of time to be nerd-sniped, and then to check in and re-prioritise
- Plausibly, focusing on RL problems is a bad starter project if you're new to ML/mech interp. I expect there to be a bunch of additional complexities, and there's less prior work that you can easily build upon. The difficulty ratings here are pretty rough and somewhat scaled to the section - **I expect a B problem here to be harder than a B elsewhere!**
 - But if you feel excited about RL specifically, don't let me discourage you!
- There's a lot of RL based algorithms in use, but my impression is that [policy gradient based algorithms](#), especially [Proximal Policy Optimization \(PPO\)](#), are the main ones

used on modern language models, and are what I'm personally most interested in understanding.

Resources

- The main two papers I've seen trying to reverse engineer RL systems are:
 - [Understanding RL vision](#) which reverse engineers a convnet trained on CoinRun (ie it directly sees the pixels of a procedurally generated Mario-style game)
 - [Acquisition of Chess Knowledge in AlphaZero](#)
 - There's some [accompanying visualizations](#)
 - [A good tutorial on the AlphaZero algorithm](#)
 - (I am not familiar with this field, and I know the lead authors of both these papers. This is definitely *not* a comprehensive literature review!)
- Learning RL:
 - [Lilian Weng's post on RL](#), a good overview
 - [Spinning Up in Deep RL](#), a more in-depth tutorial
 - [David Silver's intro to RL course](#) (creator of AlphaGo and AlphaZero)
- My [TransformerLens library](#)
 - Out of the box, this works for GPT-style language models, and will take some effort to adapt to another architecture.
 - But my guess is that the design principles used will transfer well to other architectures (eg ConvNets, MLPs, etc), and the HookPoint functionality will be a good interface for doing mech interp work (caching and editing arbitrary activations).

Problems

[This spreadsheet](#) lists each problem in the sequence. You can write down your contact details if you're working on any of them and want collaborators, see any existing work or reach out to other people on there! (thanks to Jay Bailey for making it)

- **B-C* 8.1** - Replicate some of [Tom McGrath's AlphaZero work](#) with [LeelaChessZero](#). I'd start by using NMF on the activations and trying to interpret some of these. See visualisations from the paper [here](#).

- **C-D* 8.2** - Try applying this to an open source AlphaZero style Go playing agent, and see if you can get any traction.
- **C-D* 8.3** - Train a small AlphaZero model on a simple game (eg Tic-Tac-Toe), and see if you can apply this work there. (Warning: I expect even just training the models here will be hard! Check out [this tutorial on AlphaZero](#))
- **D* 8.4** - Can you extend the work on LeelaZero? In particular, can you find anything about *how* a feature is computed? I'd start by looking for features near the start or end of the network.
- **B-C* 8.5** - Interpret one of the examples in the goal misgeneralisation papers ([Langosco et al](#) & [Shah et al](#)) - can you concretely figure out what's going on?
 - **B* 8.6** - Possible starting point: The Tree Gridworld and Monster Gridworld from [Shah et al](#) are both tiny networks, can you interpret those?
 - **C* 8.7** - CoinRun is another promising place to start. [Interpreting RL Vision](#) made significant progress interpreting an agent trained to play it, and Langosco et al found that it was an example of goal misgeneralisation - can you build on these techniques to predict the misgeneralisation
 - [CoinRun](#) is a procedurally generated Mario-style platformer, where the agent learns to get a coin at the end of the level. The coin is always in the same position during training, but Langosco et al moved it to a random position. The model could either get confused, go to the new coin or go to the old position, and they found that it still goes to the original position but otherwise plays the game correctly.
- **B-C* 8.8** - Can you apply transformer circuits techniques to a [decision transformer](#)? What do you find? (Check out [their codebase](#) and [HuggingFace's implementation](#))
 - **B*** Try training a 1L transformer on a toy problem, like finding the shortest path in a graph, as described in the introduction
 - I recommend training a model on the easiest tasks given in the paper, but with 1L or 2L, and seeing if it can get decent performance (the ones in the paper are only 3L!)
- **B-C* 8.9** - Train and interpret a model from the [In-Context Reinforcement Learning and Algorithmic Distillation](#) paper. They trained small transformers where they input a sequence of moves for a “novel” RL task and the model output sensible answers *for that task*
 - The paper uses 4L transformers! I bet you can get away with smaller.

- See [Sam Marks'](#) arguments for ways the paper can be overhyped or misunderstood.
- Interpreting transformers trained with [Reinforcement Learning from Human Feedback](#)
 - **D* 8.10** - Go and interpret CarperAI's RLHF model (forthcoming). What's up with that? How is it different from a vanilla language model? I expect the ideas in the concrete circuits in LLMs section will apply well here.
 - C 8.11 - Can you find any circuits corresponding to longer term planning?
 - **C* 8.12** - Can you get any traction on interpreting its reward model? I'm not aware of much work here and expect there's a lot of low hanging fruit - even just entering in text and qualitatively exploring its behaviour is a good place to start.
 - **D* 8.13** - Train a toy RLHF model, eg a one or two layer model, to do a simple task. Getting human data is expensive, so you can use a larger language model (GPT-2 XL or GPT-J is probably good enough, GPT-3 is definitely enough). Then try to interpret it.
 - Note - RL is hard and language model training is hard, so be prepared for training it *yourself* to be a massive slog, let alone interpreting it. But I would be *super* excited to see these results!
 - Do the above but for larger models, which will be necessary to train it for more interesting tasks. eg GPT-2 Medium or GPT-2 XL or GPT-J - bigger models will be much more of a pain to train but more interesting to interpret
- C 8.14 - Try training and interpreting a small model from [Guez et al](#). They trained agents with model-free RL, and showed some evidence that they'd spontaneously learned planning. Can you find any evidence for or against this?
- Can you interpret a small model trained with [policy gradients](#) on:
 - Tips:
 - I recommend picking a task with *some* non-trivial reasoning required - eg Cartpole is probably too trivial to be interesting
 - Any of a transformer, MLP and ConvNet feel like reasonable
 - B 8.15 - A gridworld task
 - **B-C* 8.16** - An [OpenAI gym](#) task
 - **C* 8.17** - An Atari game, eg Pong
 - B-D 8.18 - Try any of the above, training with Q-Learning instead.

- **B-D* 8.19** - On any of the above tasks, take an agent trained with RL, and train another network to copy the output logits of that agent. Try to reverse engineer that cloned model. Can you find the resulting circuits in the original model?
 - My *guess* is that the underlying circuits will be similar, but that the training dynamics of RL will leave lots of “vestigial organs”, that were useful in early training but now obsolete. And that the cloned system won’t have these, and so may be cleaner.
 - You can try making the cloned system smaller, which may make it easier to interpret!
- **B-D* 8.20** - On any of the above tasks, once you’ve got some traction understanding the fully trained agent, try to extend this understanding to study it during training. Can you get any insight into what’s actually going on? (See the relevant thoughts in the tips section)
 - Even just making some janky metrics for the eventual behaviour/circuit and plotting it over training would be interesting!
 - A good place to start would be to pick an intermediate checkpoint and to just try re-running all of your analysis on it and see what is consistent and what breaks.
 - I’m particularly interested in the interplay between the policy and value networks in [policy gradient algorithms](#).
- **A-D* 8.21** - Choose your own adventure! There’s a lot of work and papers in RL, pointing in a lot of interesting directions. Pick one you’re excited about, and see if you can make any progress reverse-engineering an agent studied there.

Previous:

200 COP in MI: Image Model Interpretability

1 comments 18 karma

Next:

200 COP in MI: Studying Learned Features in Language Models

2 comments 24 karma

[Log in to save where you left off](#)

Mentioned in

- 41 200 Concrete Open Problems in Mechanistic Interpretability: Introduction
- 25 Pacing Outside the Box: RNNs Learn to Plan in Sokoban
- 14 Why almost every RL agent does learned optimization
- 9 Exploratory Analysis of RLHF Transformers with TransformerLens

More from Neel Nanda

19	Building and evaluating model diffing agents	bilalchughtai, Josh Engels, Neel Nanda	6h	0
32	Models May Behave Worse When Eval Aware...	Senthooran Rajamanoharan, Neel ...	2d	0
65	models have some pretty funny attractor sta...	aryaj, Senthooran Rajamanoharan, ...	3mo	0

View more

Curated and popular this week

39	Sympathy for both sides of the egregious misalignment debate	Steven Byrnes	7h	5
26	A Mike’s-Eye View of ARC’s Research	Mikewins	3d	0
19	Building and evaluating model diffing agents	bilalchughtai, Josh Engels, Neel Nanda	6h	0